

Capa de Aplicacion

Un protocolo de la capa de aplicación define cómo los procesos de una aplicación, que se ejecutan en distintos sistemas terminales, se pasan los mensajes entre sí. En particular, un protocolo de la capa de aplicación define:

- Los tipos de mensajes intercambiados; por ejemplo, mensajes de solicitud y mensajes de respuesta.
- La sintaxis de los diversos tipos de mensajes, es decir, los campos de los que consta el mensaje y cómo se delimitan esos campos.
- La semántica de los campos, es decir, el significado de la información contenida en los campos.
- Las reglas para determinar cuándo y cómo un proceso envía mensajes y responde a los mismos.

HTTP

El corazón de la Web lo forma el Protocolo de transferencia de hipertexto (HTTP, HyperText Transfer Protocol), que es el protocolo de la capa de aplicación de la Web.

HTTP se implementa mediante dos programas: un programa cliente y un programa servidor. Ambos programas diferentes, se comunican entre sí intercambiando mensajes HTTP. HTTP define la estructura de estos mensajes y cómo el cliente y el servidor intercambian los mensajes. HTTP define cómo los clientes web solicitan páginas web a los servidores y cómo estos servidores web transfieren esas páginas a los clientes.

Utiliza TCP como su protocolo de transporte (en lugar de ejecutarse por encima de UDP). El cliente HTTP primero inicia una conexión TCP con el servidor. Una vez que la conexión se ha establecido, los procesos de navegador y de servidor acceden a TCP a través de sus interfaces de socket.

TCP proporciona un servicio de transferencia de datos fiable a HTTP. Esto implica que cada mensaje de solicitud HTTP enviado por un proceso cliente llegará intacto al servidor; del mismo modo, cada mensaje de respuesta HTTP enviado por el proceso servidor llegará intacto al cliente. HTTP no tiene que preocuparse por las pérdidas de datos o por los detalles sobre cómo TCP recupera los datos perdidos o los reordena

dentro de la red. Ése es el trabajo de TCP y de los protocolos de las capas inferiores de la pila de protocolos.

Es importante observar que el servidor envía los archivos solicitados a los clientes sin almacenar ninguna información acerca del estado del cliente. Si un determinado cliente pide el mismo objeto dos veces en un espacio de tiempo de unos pocos segundos, el servidor no responde diciendo que acaba de servir dicho objeto al cliente; en su lugar, el servidor reenvía el objeto. Dado que un servidor HTTP no mantiene ninguna información acerca de los clientes, se dice que HTTP es un protocolo sin memoria del estado.

Conexiones no persistentes

Proceso de HTTP no persistente:

1. El proceso cliente HTTP inicia una conexión TCP con el servidor en el puerto número 80, que es el número de puerto predeterminado para HTTP. Asociados con la conexión TCP, habrá un socket en el cliente y un socket en el servidor.
2. El cliente HTTP envía un mensaje de solicitud HTTP al servidor a través de su socket. El mensaje de solicitud incluye el nombre de la ruta.
3. El proceso servidor HTTP recibe el mensaje de solicitud a través de su socket, recupera el objeto de su medio de almacenamiento (RAM o disco), encapsula el objeto en un mensaje de respuesta HTTP y lo envía al cliente a través de su socket.
4. El proceso servidor HTTP indica a TCP que cierre la conexión TCP. (Pero TCP realmente no termina la conexión hasta que está seguro de que el cliente ha recibido el mensaje de respuesta en perfecto estado.)
5. El cliente HTTP recibe el mensaje de respuesta. La conexión TCP termina. El mensaje indica que el objeto encapsulado es un archivo HTML. El cliente extrae el archivo del mensaje de respuesta, examina el archivo HTML y encuentra las referencias a los objetos.
6. Los cuatro primeros pasos se repiten entonces para cada uno de los objetos referenciados.

Las conexiones no persistentes presentan algunos inconvenientes. En primer lugar, tiene que establecerse y gestionarse una conexión completamente nueva para cada

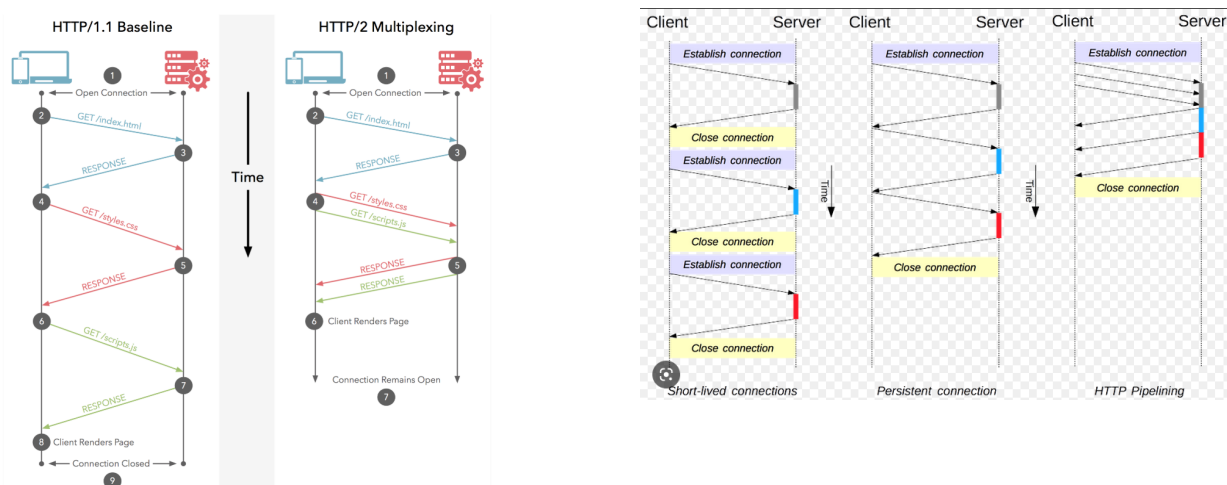
objeto solicitado. Para cada una de estas conexiones, deben asignarse buffers TCP y tienen que gestionarse variables TCP tanto en el cliente como en el servidor. Esto puede sobrecargar al servidor web, ya que puede estar sirviendo solicitudes de cientos de clientes distintos simultáneamente. En segundo lugar cada objeto sufre un retardo de entrega para establecer la conexión y para solicitar y recibir el objeto.

Conexiones persistentes

Con las conexiones persistentes de HTTP 1.1, el servidor deja la conexión TCP abierta después de enviar una respuesta. Las subsiguientes solicitudes y respuestas que tienen lugar entre el mismo cliente y el servidor pueden enviarse a través de la misma conexión.

Estas solicitudes de objetos pueden realizarse una tras otra sin esperar a obtener las respuestas a las solicitudes pendientes (pipelining, procesamiento en cadena). Normalmente, el servidor HTTP cierra una conexión cuando no se ha utilizado durante cierto tiempo (un intervalo de fin de temporización configurable). Cuando el servidor recibe las solicitudes una tras otra, envía los objetos uno tras otro. El modo predeterminado de HTTP utiliza conexiones persistentes con procesamiento en cadena.

En HTTP/2 se amplía la funcionalidad, permitiendo entrelazar múltiples solicitudes y respuestas en la misma conexión, y proporcionando también un mecanismo para priorizar los mensajes de solicitud y respuesta HTTP dentro de dicha conexión.



Formato mensaje de solicitud

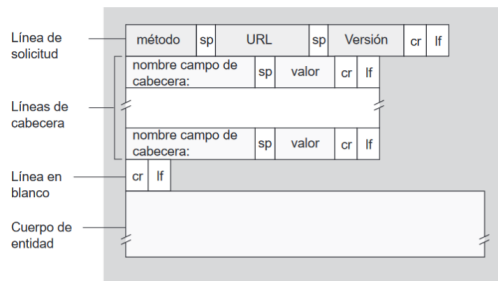


Figura 2.8 ♦ Formato general de un mensaje de solicitud HTTP.

Mensaje de solicitud HTTP

A continuación se muestra un mensaje de solicitud HTTP típico:

```
GET /unadireccion/pagina.html HTTP/1.1
Host: www.unaEscuela.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

La primera línea de un mensaje de solicitud HTTP se denomina línea de solicitud y las siguientes son las líneas de cabecera. La línea de solicitud consta de tres campos: el campo de método, el campo URL y el campo de la versión HTTP. El campo que especifica el método puede tomar diferentes valores, entre los que se incluyen GET, POST, HEAD, PUT y DELETE. La inmensa mayoría de los mensajes de solicitud HTTP utilizan el método GET

La línea de cabecera Host: especifica el host en el que reside el objeto.

Al incluir la línea de cabecera Connection: close, el navegador está diciendo al servidor que no desea molestarse en trabajar con conexiones persistentes, sino que desea que el servidor cierre la conexión después de enviar el objeto solicitado.

La línea de cabecera User-agent: especifica el agente de usuario, es decir, el tipo de navegador que está haciendo la solicitud al servidor. Esta línea de cabecera resulta útil porque el servidor puede enviar versiones diferentes del mismo objeto a los distintos tipos de agentes de usuario la línea de cabecera Accept-language: indica que el usuario prefiere recibir una versión en francés del objeto, si tal objeto existe en el servidor; en caso contrario, el servidor debe enviar la versión predeterminada.

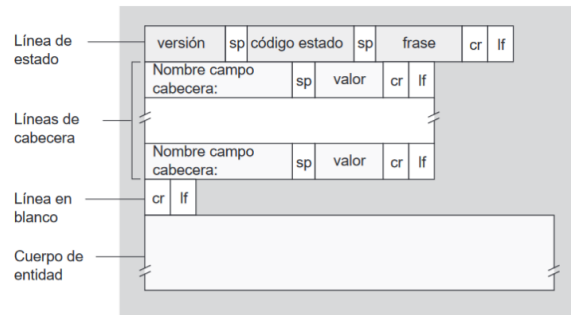
La línea de cabecera Accept-language: sólo es una de las muchas cabeceras de negociación del contenido disponibles en HTTP.

Formato mensaje de Respuesta

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
```

```
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(datos datos datos datos datos ...)
```



Tiene tres secciones: una línea de estado inicial, seis líneas de cabecera y después el cuerpo de entidad. El cuerpo de entidad es la parte más importante del mensaje, ya que contiene el objeto solicitado en sí

La línea de estado contiene tres campos: el que especifica la versión del protocolo {HTTP/1.1 en este caso}, un código de estado {200} y el correspondiente mensaje explicativo del estado {OK}.

El servidor utiliza la línea de cabecera Connection: close para indicar al cliente que va a cerrar la conexión TCP después de enviar el mensaje.

La línea de cabecera Date: indica la hora y la fecha en la que se creó la respuesta HTTP y fue enviada por el servidor. Observe que esta línea es la hora en la que el servidor recupera el objeto de su sistema de archivos, inserta el objeto en el mensaje de respuesta y lo envía.

La línea de cabecera Server: indica que el mensaje fue generado por un servidor web Apache; es análoga a la línea de cabecera User-agent: del mensaje de solicitud HTTP.

La línea de cabecera Last-Modified: especifica la hora y la fecha en que el objeto fue creado o modificado por última vez. Esta línea resulta fundamental para el almacenamiento en caché del objeto, tanto para el cliente local como en los servidores de almacenamiento en caché de la red

La línea de cabecera Content-Length: especifica el número de bytes del objeto que está siendo enviado.

La línea Content-Type: indica que el objeto incluido en el cuerpo de entidad es texto HTML.

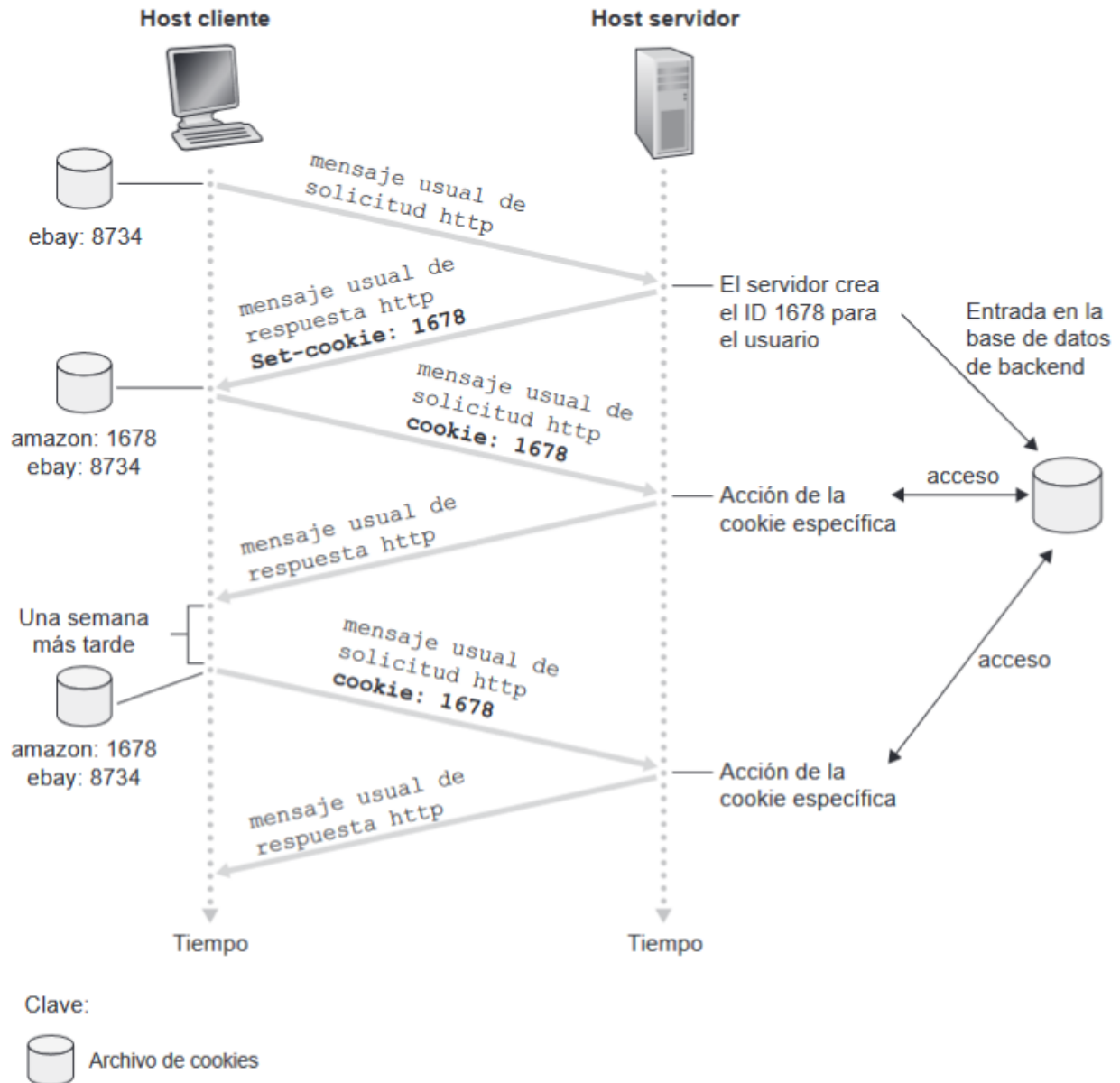
Cookies

HTTP no tiene memoria del estado de la conexión. Esto simplifica el diseño del servidor y permite desarrollar servidores web de alto rendimiento que pueden gestionar miles de conexiones TCP simultáneas. Sin embargo, para un sitio web, a menudo es deseable poder identificar a los usuarios, bien porque el servidor desea restringir el acceso a los usuarios o porque desea servir el contenido en función de la identidad del usuario. Para estos propósitos, HTTP utiliza cookies. Las cookies, definidas en permiten a los sitios seguir la pista a los usuarios.

La tecnología de las cookies utiliza cuatro componentes:

- (1) Una línea de cabecera de la cookie en el mensaje de respuesta HTTP;
- (2) una línea de cabecera de la cookie en el mensaje de solicitud HTTP;
- (3) el archivo de cookies almacenado en el sistema terminal del usuario y gestionado por el navegador del usuario;
- (4) una base de datos back-end en el sitio web.

Las cookies pueden utilizarse para identificar a un usuario. La primera vez que un usuario visita un sitio, el usuario puede proporcionar una identificación suya. En las sesiones posteriores, el navegador pasa al servidor una cabecera de cookie, identificando al usuario ante el servidor. Las cookies pueden por tanto utilizarse para crear una capa de sesión por encima del protocolo HTTP sin memoria del estado de la conexión.



Cache web

Una caché web, también denominada servidor proxy, es una entidad de red que satisface solicitudes HTTP en nombre de un servidor web de origen. La caché web dispone de su propio almacenamiento en disco y mantiene en él copias de los objetos solicitados recientemente.

1. El navegador establece una conexión TCP con la caché web y envía una solicitud HTTP para el objeto a la caché web.

2. La caché web comprueba si tiene una copia del objeto almacenada localmente. Si la tiene, la caché web devuelve el objeto dentro de un mensaje de respuesta HTTP al navegador del cliente.

3. Si la caché web no tiene el objeto, abre una conexión TCP con el servidor de origen. La caché web envía entonces una solicitud HTTP para obtener el objeto a través de la conexión TCP caché-servidor. Después de recibir esta solicitud, el servidor de origen envía el objeto dentro de un mensaje de respuesta HTTP a la caché web.

4. Cuando la caché web recibe el objeto, almacena una copia en su dispositivo de almacenamiento local y envía una copia, dentro de un mensaje de respuesta HTTP, al navegador del cliente (a través de la conexión TCP existente entre el navegador del cliente y la caché web).

Habitualmente es un ISP quien adquiere e instala una caché web.

El almacenamiento en caché web es útil por dos razones. La primera razón es que una caché web puede reducir sustancialmente el tiempo de respuesta a la solicitud de un cliente. La segunda razón es que las cachés web pueden reducir sustancialmente el tráfico en el enlace de acceso a Internet de una institución.

Además, las cachés web pueden reducir mucho el tráfico web global en Internet, mejorando en consecuencia el rendimiento de todas las aplicaciones.

Get condicional

Aunque el almacenamiento en caché puede reducir los tiempos de respuesta percibidos por el usuario, introduce un nuevo problema: la copia de un objeto que reside en la caché puede estar desactualizada.

HTTP dispone de un mecanismo que permite a la caché verificar que sus objetos están actualizados. Este mecanismo se denomina GET condicional. Un mensaje de solicitud HTTP se denomina también mensaje GET condicional si (1) el mensaje de solicitud utiliza el método GET y (2) el mensaje de solicitud incluye una línea de cabecera *If-Modified-Since*:

Correo electrónico

Existen tres componentes principales: agentes de usuario, servidores de correo y el Protocolo simple de transferencia de correo

Los servidores de correo forman el núcleo de la infraestructura del correo electrónico. Cada destinatario, como por ejemplo Benito, tiene un buzón de correo ubicado en uno de los servidores de correo. El buzón de Benito gestiona y mantiene los mensajes que le han sido enviados. Un mensaje típico inicia su viaje en el agente de usuario del emisor, viaja hasta el servidor de correo del emisor y luego hasta el servidor de correo del destinatario, donde es depositado en el buzón del mismo. Cuando Benito quiere acceder a los mensajes contenidos en su buzón, el servidor de correo que lo contiene autentica a Benito (mediante el nombre de usuario y la contraseña). El servidor de correo de Alicia también tiene que ocuparse de los fallos que se producen en el servidor de correo de Benito. Si el servidor de Alicia no puede enviar el mensaje de correo al servidor de Benito, entonces el servidor de Alicia mantiene el mensaje en una cola de mensajes e intenta enviarlo más tarde.

SMTP es el principal protocolo de la capa de aplicación para el correo electrónico por Internet. Utiliza el servicio de transferencia de datos fiable de TCP para transferir el correo desde el servidor de correo del emisor al servidor de correo del destinatario.

Al igual que la mayoría de los protocolos de la capa de aplicación, SMTP tiene dos lados: el lado del cliente, que se ejecuta en el servidor de correo del emisor, y el lado del servidor, que se ejecuta en el servidor de correo del destinatario. Tanto el lado del cliente como el del servidor de SMTP se ejecutan en todos los servidores de correo.

El protocolo SMTP es el corazón del correo electrónico por Internet. SMTP transfiere mensajes desde los servidores de correo de los emisores a los servidores de correo de los destinatarios.

Para ilustrar el funcionamiento básico de SMTP vamos a recurrir a un escenario ya conocido. Suponga que Alicia desea enviar a Benito un sencillo mensaje ASCII.

1. Alicia invoca a su agente de usuario para correo electrónico, proporciona la dirección de correo electrónico de Benito (por ejemplo, `benito@unaescuela.edu`), compone un mensaje e indica al agente de usuario que lo envíe.
2. El agente de usuario de Alicia envía el mensaje al servidor de correo de ella, donde es colocado en una cola de mensajes.
3. El lado del cliente de SMTP, que se ejecuta en el servidor de correo de Alicia, ve el mensaje en la cola de mensajes. Abre una conexión TCP con un servidor SMTP, que se ejecuta en el servidor de correo de Benito.

4. Después de la fase de negociación inicial de SMTP, el cliente SMTP envía el mensaje de Alicia a través de la conexión TCP.
5. En el servidor de correo de Benito, el lado del servidor de SMTP recibe el mensaje. El servidor de correo de Benito coloca entonces el mensaje en el buzón de Benito.
6. Benito invoca a su agente de usuario para leer el mensaje cuando le apetezca.

Normalmente SMTP no utiliza servidores de correo intermedios para enviar correo, incluso cuando los dos servidores de correo se encuentran en extremos opuestos del mundo.

Veamos en detalle cómo transfiere SMTP un mensaje desde un servidor de correo emisor a un servidor de correo receptor. Comprobaremos que el protocolo SMTP presenta muchas similitudes con los protocolos empleados por las personas para las interacciones cara a cara.

- El cliente SMTP (que se ejecuta en el host servidor de correo emisor) establece una conexión TCP con el puerto 25 del servidor SMTP (que se ejecuta en el host servidor de correo receptor). Si el servidor no está operativo, el cliente lo intentará más tarde.
- Una vez que se ha establecido la conexión, el servidor y el cliente llevan a cabo el proceso de negociación de la capa de aplicación (al igual que las personas, que antes de intercambiar información se presentan, los clientes y servidores SMTP se presentan a sí mismos antes de transferir la información). Durante esta fase de negociación SMTP, el cliente SMTP especifica la dirección de correo electrónico del emisor (la persona que ha generado el mensaje) y la dirección de correo electrónico del destinatario.
- Una vez que el cliente y el servidor SMTP se han presentado a sí mismos, el cliente envía el mensaje. SMTP cuenta con el servicio de transferencia de datos fiable de TCP para transferir el mensaje al servidor sin errores.
- El cliente repite entonces este proceso a través de la misma conexión TCP si tiene que enviar otros mensajes al servidor; en caso contrario, indica a TCP que cierre la conexión.

SMTP vs HTTP

Ambos protocolos se emplean para transferir archivos de un host a otro: HTTP transfiere archivos (también denominados objetos) desde un servidor web a un cliente web (normalmente, un navegador); SMTP transfiere archivos (es decir, mensajes de correo electrónico) desde un servidor de correo a otro servidor de correo.

Para transferir los archivos, tanto HTTP persistente como SMTP emplean conexiones persistentes. Sin embargo HTTP es un protocolo pull porque se usa para obtener cosas y SMTP es push porque se usa para mandar cosas.

Una segunda diferencia es que SMTP requiere que cada mensaje, incluyendo el cuerpo de cada mensaje, esté en el formato ASCII de 7 bits. Si el mensaje contiene caracteres que no corresponden a dicho formato o contiene datos binarios (como por ejemplo un archivo de imagen), entonces el mensaje tiene que ser codificado en ASCII de 7 bits. Los datos HTTP no imponen esta restricción.

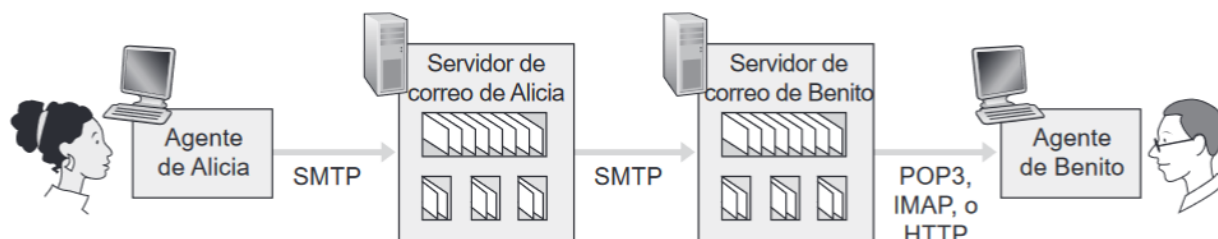
Una tercera diferencia es que HTTP encapsula cada objeto en su propio mensaje de respuesta HTTP. El correo de Internet incluye todos los objetos del mensaje en un mismo mensaje.

Cabecera de mensaje

Toda cabecera tiene que estar formada por una línea de cabecera From: y una línea de cabecera To:; también puede incluir una línea Subject:, así como otras líneas de cabecera opcionales.

Extraccion de mensajes

Como corno saca benito un mensaje del servidor de correo? Facil inventamos otro protocolo más porque en redes nos encantan los protocolos. Entonces como SMTP solo puede meterla, inventamos otro protocolo (IMAP y POP3) para sacarla.



POP3

POP3 es un protocolo de acceso a correo extremadamente simple.

Dado que el protocolo es tan simple, su funcionalidad es bastante limitada. POP3 se inicia cuando el agente de usuario (el cliente) abre una conexión TCP en el puerto 110 al servidor de correo (el servidor). Una vez establecida la conexión TCP, POP3 pasa a través de tres fases: autorización, transacción y actualización.

Durante la primera fase, la autorización, el agente de usuario envía un nombre de usuario y una contraseña (en texto legible) para autenticar al usuario.

Durante la segunda fase, la de transacción, el agente de usuario recupera los mensajes; también durante esta fase, el agente de usuario puede marcar los mensajes para borrado, eliminar las marcas de borrado y obtener estadísticas de correo.

La tercera fase, la actualización, tiene lugar después que el cliente haya ejecutado el comando quit, terminando la sesión POP3; en este instante, el servidor de correo borra los mensajes que han sido marcados para borrado.

En una transacción POP3, el agente de usuario ejecuta comandos y el servidor devuelve para cada comando una respuesta. Existen dos posibles respuestas: +OK (seguida en ocasiones por una serie de datos servidor-cliente), utilizada por el servidor para indicar que el comando anterior era correcto; y -ERR, utilizada por el servidor para indicar que había algún error en el comando anterior.

La fase de autorización tiene dos comandos principales: *user <nombreusuario>* y *pass <contraseña>*.

Un agente de usuario que utilice POP3 suele ser configurado (por el usuario) para “descargar y borrar” o para “descargar y guardar”. En el modo descargar y borrar, el agente de usuario ejecutará los comandos list, retr y dele. En primer lugar, el agente de usuario pide al servidor de correo que le informe del tamaño de cada uno de los mensajes almacenados. A continuación, el agente de usuario recupera y borra cada uno de los mensajes del servidor.

Un problema en este modo de descarga y borrado es que el destinatario, Benito, puede ser algo nómada y puede desear poder acceder a sus mensajes de correo desde varias máquinas; por ejemplo, desde el PC de la oficina, el PC de su casa y el portátil. El modo de descarga y borrado reparte los mensajes de correo de Benito entre estas tres máquinas; en particular, si Benito lee un mensaje en el PC de la oficina, no podrá volver a leerlo en el portátil en su casa por la noche. En el modo descargar y guardar, el agente de usuario deja los mensajes en el servidor de correo después de que se hayan

descargado. En este caso, Benito podrá volver a leer los mensajes desde distintas máquinas; puede acceder a un mensaje en el trabajo y luego también en su casa si lo desea.

IMAP

IMAP es un protocolo de acceso a correo. Ofrece muchas más funcionalidades que POP3, pero también es significativamente más complejo

Un servidor IMAP asociará cada mensaje con una carpeta; cuando un mensaje llega al servidor, se asocia con la carpeta INBOX (Bandeja de entrada) del destinatario, el cual puede entonces pasar el mensaje a una nueva carpeta creada por el usuario, leer el mensaje, borrarlo, etc. El protocolo IMAP proporciona comandos que permiten a los usuarios crear carpetas y mover los mensajes de una carpeta a otra. IMAP también proporciona comandos que permiten a los usuarios realizar búsquedas en carpetas remotas para localizar mensajes que cumplan unos determinados criterios.

A diferencia de POP3, un servidor IMAP mantiene información acerca del estado a lo largo de las sesiones IMAP, como por ejemplo, los nombres de las carpetas y los mensajes asociados con cada una de ellas.

Otra característica de IMAP es que dispone de comandos que permiten a un agente de usuario obtener partes componentes de los mensajes. Esta característica resulta útil cuando se emplea una conexión con un ancho de banda pequeño (por ejemplo, un enlace de módem de baja velocidad) entre el agente de usuario y su servidor de correo.

Con una conexión de ancho de banda pequeño, puede que el usuario no quiera descargar todos los mensajes guardados en su buzón y que desee saltarse los mensajes largos que pueda haber; por ejemplo un archivo de audio o de vídeo.

DNS

Los hosts pueden tener 2 formas de identificarse, la primera puede ser por su propio nombre de hosts (ww.facebook.com) y otra es la dirección IP, la cual es de una longitud fija y única. Las personas suelen usar el nombre de host mientras que los routers prefieren usar IP.

Para reconciliar estas preferencias necesitamos un servicio de directorio que traduzca los nombres de host en direcciones IP. Esta es la tarea principal que lleva a cabo el Sistema de nombres de dominio (DNS, Domain Name System) de Internet.

DNS es (1) una base de datos distribuida implementada en una jerarquía de servidores DNS y (2) un protocolo de la capa de aplicación que permite a los hosts consultar la base de datos distribuida.

Otros protocolos de la capa de aplicación, como HTTP y SMTP, emplean habitualmente DNS para traducir los nombres de host suministrados por el usuario en direcciones IP.

Por ejemplo, veamos qué ocurre cuando un navegador (es decir, un cliente HTTP), que se ejecuta en un determinado host de usuario, solicita el URL `www.unaescuela.edu/index.html`.

Para que el host del usuario pueda enviar un mensaje de solicitud HTTP al servidor web `www.unaescuela.edu`, el host del usuario debe obtener en primer lugar la dirección IP de `www.unaescuela.edu`. Esto se hace del siguiente modo:

1. La propia máquina cliente ejecuta el lado del cliente de la aplicación DNS.
2. El navegador extrae el nombre de host, `www.unaescuela.edu`, del URL y pasa el nombre de host al lado del cliente de la aplicación DNS.
3. El cliente DNS envía una consulta que contiene el nombre de host a un servidor DNS.
4. El cliente DNS recibe finalmente una respuesta, que incluye la dirección IP correspondiente al nombre del host.
5. Una vez que el navegador recibe la dirección IP del servidor DNS, puede iniciar una conexión

TCP con el proceso servidor HTTP localizado en el puerto 80 en esa dirección IP.

Podemos ver a partir de este ejemplo que DNS añade un retardo adicional, en ocasiones, sustancial,

a las aplicaciones de Internet que le utilizan.

DNS proporciona algunos otros servicios importantes además de la traducción de los nombres

de host en direcciones IP:

▼ Alias de host

Un host con un nombre complicado puede tener uno o más alias. El nombre de host base, el cual posee diferentes alias es al que se denomina host canonico. Una aplicación puede invocar DNS para obtener el nombre de host canónico para un determinado alias, así como la dirección IP del host.

▼ Alias del servidor de correo

Los servidores de correo suelen tener alias para identificar las cuentas de mail de sus usuarios. Una aplicación de correo puede invocar al servicio DNS para obtener el nombre de host canónico para un determinado alias, así como la dirección IP del host.

▼ Distribucion de cargas

DNS también se emplea para realizar la distribución de carga entre servidores replicados, como los servidores web replicados. Los sitios con una gran carga de trabajo, como cnn.com, están replicados en varios servidores, ejecutándose cada servidor en un sistema terminal distinto y teniendo cada uno una dirección IP diferente. Para los servidores web replicados hay asociado un conjunto de direcciones IP con un único nombre de host canónico. La base de datos DNS contiene este conjunto de direcciones IP.

Estructura de DNS

DNS utiliza un gran número de servidores, organizados de forma jerárquica y distribuidos alrededor de todo el mundo. Ningún servidor DNS dispone de todas las correspondencias de todos los hosts de Internet. En su lugar, las correspondencias están repartidas por los servidores DNS.

Existen 3 clases de servidores DNS:

▼ DNS Raiz

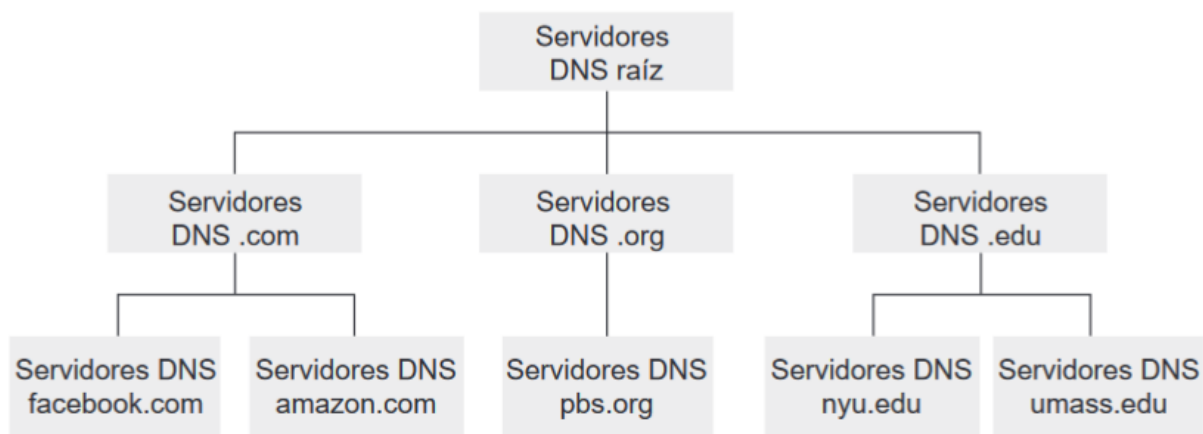
Los servidores de nombres raíz proporcionan las direcciones IP de los servidores TLD.

▼ De dominio superior

Para todos los dominios de nivel superior, como son com, org, net, edu y gov, y todos los dominios de nivel superior correspondientes a los distintos países, como por ejemplo, uk, fr, ca y jp, existe un servidor TLD. Los servidores TLD proporcionan las direcciones IP para los servidores DNS autoritativos.

▼ DNS Autoritativo

Todas las organizaciones que tienen hosts accesibles públicamente (como son los servidores web y los servidores de correo) a través de Internet deben proporcionar registros DNS accesibles públicamente que establezcan la correspondencia entre los nombres de dichos hosts y sus direcciones IP. Un servidor DNS autoritativo de una organización alberga estos registros DNS. Una organización puede elegir implementar su propio servidor DNS autoritativo para almacenar estos registros; alternatively, la organización puede pagar por tener esos registros almacenados en un servidor DNS autoritativo de algún proveedor de servicios



Existe otro tipo importante de servidor DNS conocido como servidor DNS local. Un servidor DNS local no pertenece estrictamente a la jerarquía de servidores, pero no obstante es importante dentro de la arquitectura DNS. Cada ISP (como por ejemplo un ISP residencial o un ISP institucional) tiene un servidor DNS local (también denominado servidor de nombres predeterminado). Cuando un host se conecta a un ISP, este proporciona al host las direcciones IP de uno o más de sus servidores DNS locales

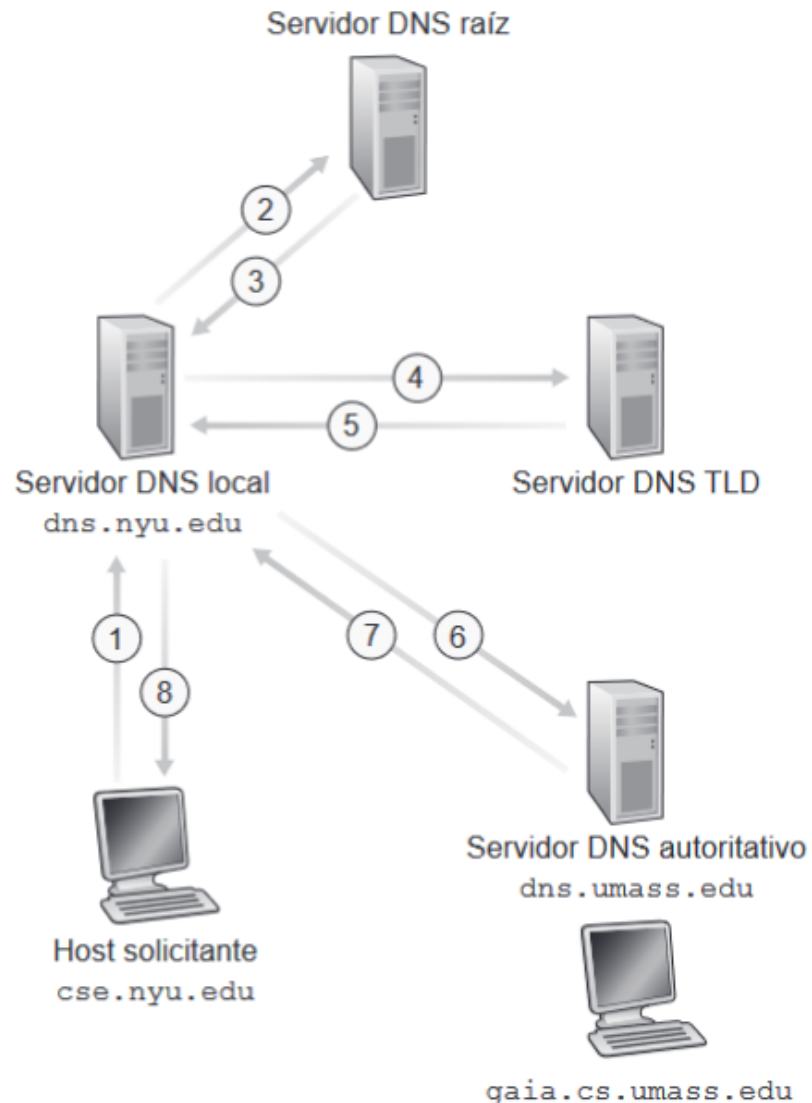


Figura 2.19 ♦ Interacción de los distintos servidores DNS.

Cache DNS

El almacenamiento en caché DNS es una funcionalidad extremadamente importante del sistema DNS. En realidad, DNS explota exhaustivamente el almacenamiento en caché para mejorar el comportamiento de los retardos y reducir el número de mensajes DNS que van de un lado a otro por Internet.

La idea que se esconde detrás del almacenamiento en caché DNS es muy simple. En una cadena de consultas, cuando un servidor DNS recibe una respuesta DNS (que

contiene, por ejemplo, una correspondencia entre un nombre de host y una dirección IP), puede almacenar esta información en su memoria local.

Si una relación nombre de host/dirección IP se almacena en la caché de un servidor DNS y llegan otras consultas a ese mismo servidor DNS preguntando por el mismo nombre de host, el servidor DNS puede proporcionar la dirección IP deseada, incluso aunque no sea autoritativo para el nombre de host. Dado que los hosts y las correspondencias entre nombres de host y direcciones IP no son permanentes, los servidores DNS descartan la información almacenada en caché pasado un cierto periodo de tiempo (normalmente, unos dos días).

Registros y mensajes DNS

Los servidores DNS que implementan conjuntamente la base de datos distribuida DNS almacenan los registros de recursos (RR), incluyendo los que proporcionan las correspondencias entre nombre de host y dirección IP. Cada mensaje de respuesta DNS transporta uno o más registros de recursos.

Un registro de recurso está formado por los siguientes cuatro campos:

(Nombre, Valor, Tipo, TTL)

TTL es el tiempo de vida del registro de recurso; determina cuándo un recurso debería ser elimi-

nado de una caché.

El significado de Nombre y Valor depende del campo Tipo:

▼ Tipo=A

Nombre es un nombre de host y Valor es la dirección IP correspondiente a dicho nombre.

Por tanto, un registro de tipo A proporciona la correspondencia estándar nombre de host-dirección IP

▼ Tipo=NS

Nombre es un dominio (como foo.com) y Valor es el nombre de host de un servidor DNS autoritativo que sabe cómo obtener las direcciones IP de los hosts del dominio.

Este registro se utiliza para enrutar las consultas DNS a lo largo de la cadena de consultas.

▼ Tipo=CNAME

Valor es un nombre de host canónico correspondiente al alias especificado por Nombre. Este registro puede proporcionar a los hosts que hacen consultas el nombre canónico correspondiente a un nombre de host.

▼ Tipo=MX

Valor es el nombre canónico de un servidor de correo que tiene un alias dado por Nombre. Los registros MX permiten a los nombres de host de los servidores de correo tener alias simples. De esta forma una empresa puede tener el mismo alias para su servidor de correo y para uno de sus otros servidores (como por ejemplo, el servidor web). Para obtener el nombre canónico del servidor de correo, un cliente DNS consultaría un registro MX y para conocer el nombre canónico del otro servidor, consultaría el registro CNAME.

Si un servidor DNS es autoritativo para un determinado nombre de host, entonces el servidor DNS contendrá un registro de tipo A para el nombre de host. (Incluso aunque el servidor DNS no sea autoritativo, puede contener un registro de tipo A en su caché.) Si un servidor no es autoritativo para un nombre de host, entonces el servidor contendrá un registro de tipo NS para el dominio que incluye el nombre de host; también contendrá un registro de tipo A que proporcione la dirección IP del servidor DNS en el campo Valor del registro NS.

Mensajes DNS

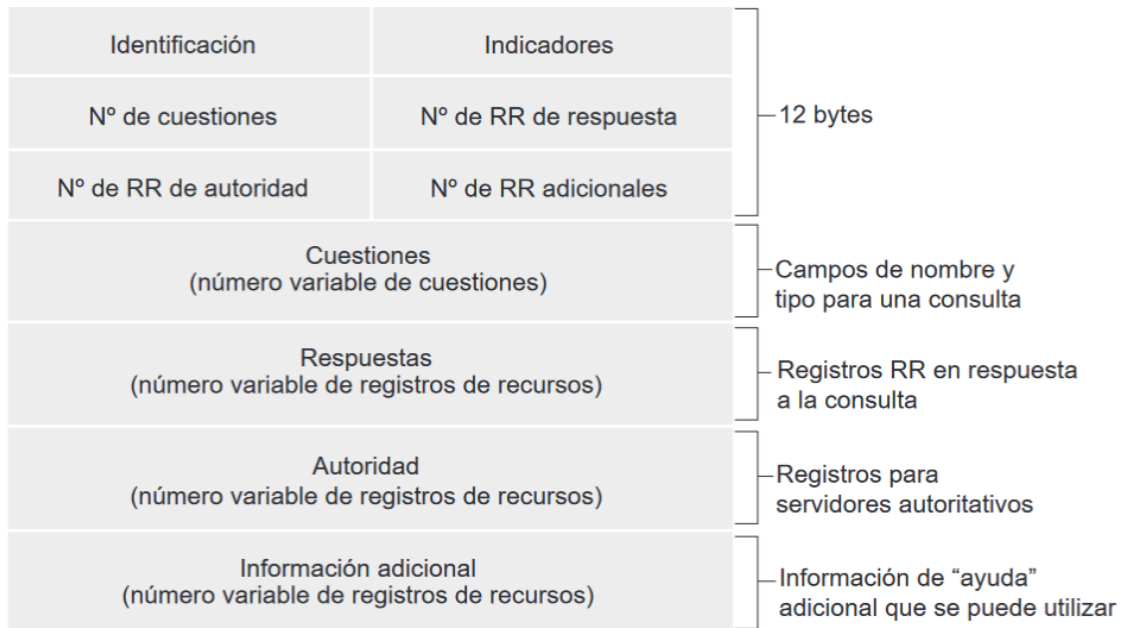


Figura 2.21 ♦ Formato de los mensajes DNS.

Las consultas DNS presentan flags, los mas importantes son:

AA: Se activa en un mensaje de respuesta cuando un servidor DNS es un servidor autoritativo para un nombre solicitado

RD: Se activa cuando un cliente desea que el servidor DNS realice una recursion cuando no disponga del registro

RA: Se activa si el servidor DNS soporta la recursion.

Bit	Flag	Description
bit 5	AA	Authoritative Answer
bit 6	TC	Truncated Response
bit 7	RD	Recursion Desired
bit 8	RA	Recursion Available
bit 9		Reserved
bit 10	AD	Authentic Data
bit 11	CD	Checking Disabled

diferentes flags posibles

- La sección cuestiones contiene información acerca de la consulta que se va a realizar. Esta sección incluye un campo de nombre que contiene el nombre que se

va a consultar y un campo de tipo que especifica el tipo de cuestión que se plantea acerca del nombre; por ejemplo, la dirección del host asociada con un nombre (tipo A) o el servidor de correo para un nombre (tipo MX).

- En una respuesta de un servidor DNS, la sección respuestas contiene los registros del recurso para el nombre que fue consultado originalmente. En cada registro de recurso existe un parámetro Tipo (por ejemplo, A, NS, CNAME y MX), un parámetro Valor y el parámetro TTL. Una respuesta puede devolver varios registros de recursos, ya que un nombre de host puede tener asociadas varias direcciones IP.
- La sección autoridad contiene registros de otros servidores autoritativos.
- La sección información adicional contiene otros registros útiles. Por ejemplo, el campo de respuesta en un mensaje de respuesta a una consulta MX contiene un registro de recurso que proporciona el nombre de host canónico de un servidor de correo. Esta sección de información adicional contiene un registro de tipo A que proporciona la dirección IP para el nombre de host canónico del servidor de correo.